

Datamanagement & Analytics

Prof. Dr. André Drews

Fachbereich Maschinenbau und Wirtschaft

13. April 2026

- nur für den Gebrauch innerhalb des Kurses -

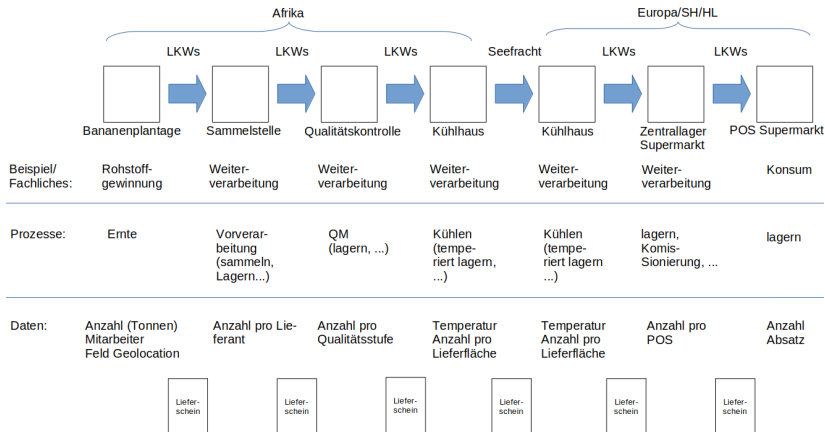
Vorlesungskapitel 3

- Datenbanken -

Lernziele

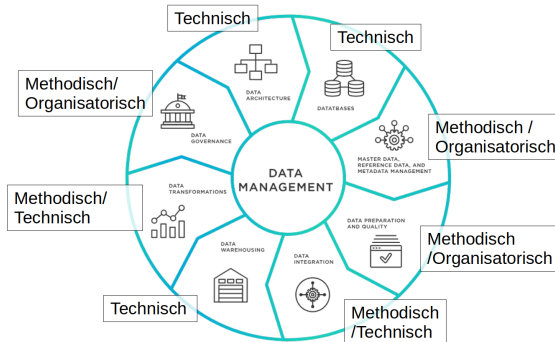
- Kompetenz im Bereich Datenbanken

1. Wiederholung
2. Relationale Datenbanken
3. Dokumentenbasierte Datenbanken



- .CSV
- .json
- .xml, .xsd
- .yaml

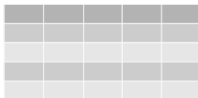
Datenmanagement Datenbanken



Structured Query Language (SQL)/ Relational

Anfang 1980er
Sehr hohe
Verbreitung

relational



No not only Structured Query Language (No - SQL)

Mitte 1960er
Nahezu komplett
abgelöst

hierar-
chisch



Ende 1980er
Niedrige
Verbreitung

objekt-
orientiert



Anfang 2000er
mittlere
Verbreitung

document
store

```
data {  
  data1 {  
    "name": "data1",  
  },  
  data2 {  
    "name": "data2",  
  }  
}
```

Mitte 2000er
Relativ niedrige
Verbreitung

graph
store



Mitte 2000er
niedrige bis mittlere
Verbreitung

wide column
stores

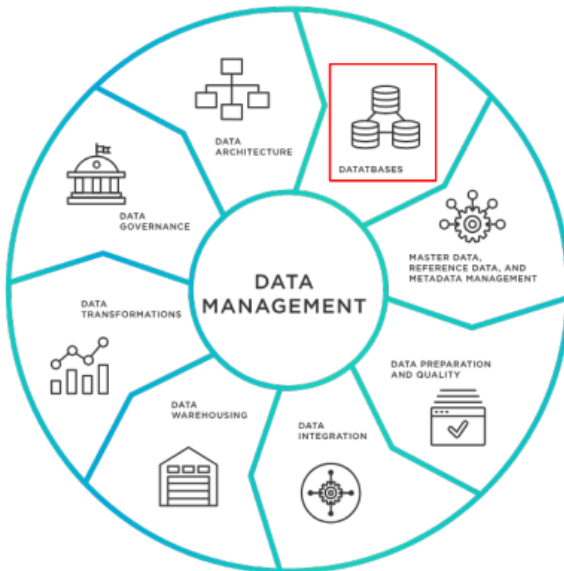


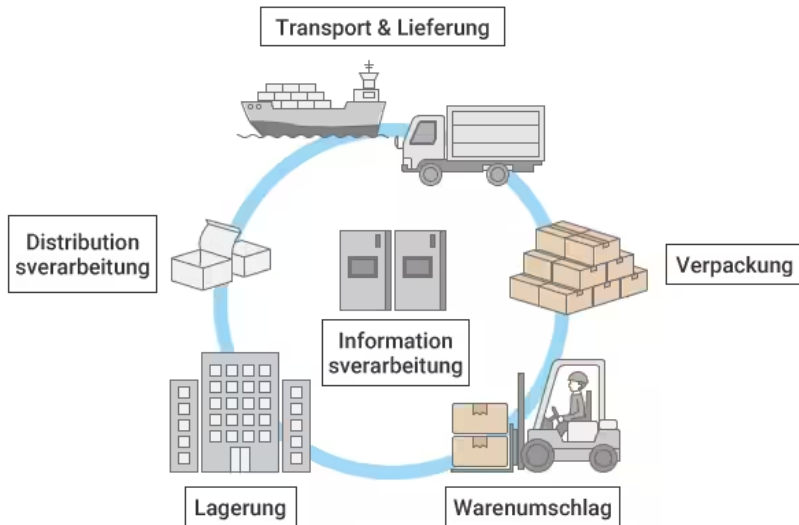
Mitte 2010er
niedrige bis mittlere
Verbreitung

key-value
store



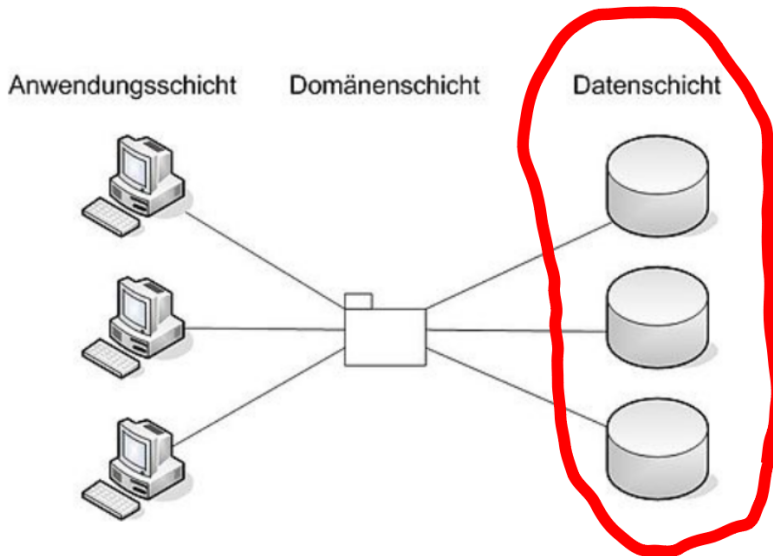
1. Überblick Datenbanken
2. Relationale Datenbanken
3. Dokumentenbasierte Datenbanken







Datenbanken in Applikations- und Informationssystemen



Datenbank oder Database

Datenbank

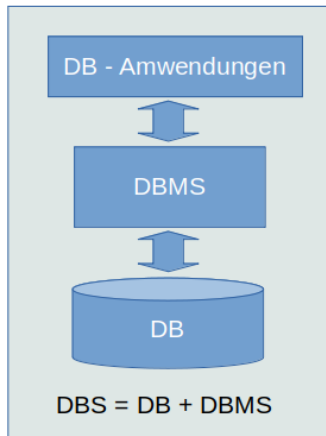
Zentrale Stelle, Einrichtung, in der Daten vorhanden sind, aufbewahrt werden, bei Bedarf entnommen oder abgerufen werden können"

- Es wird nicht immer begrifflich sauber zwischen Daten, Informationen und Wissen getrennt. Oft wird der "atomistische" Begriff Datenbank herangezogen.
- Bei Wissen wird die entsprechende "Wissensbank" auch Wissens-Datenbank bezeichnet z.B. ein Wiki (Informationssystem in Form einer Webside (Client-Server Architektur mit einer Datenbank auf dem Server, in dem die Nutzer Wissen kollektiv sammeln können)).

Datenbank

"selbstständige, auf Dauer und flexiblen und sicheren Gebrauch ausgelegte Datenorganisation, die sowohl eine Datenbasis als auch eine zugehörige Datenverwaltung - Datenbankmanagementsystem (DBMS) - umfasst. Eine Datenbank dient dazu, eine große Menge von Daten [strukturiert] zu speichern und zu verwalten."

Quelle: <https://wirtschaftslexikon.gabler.de/definition/datenbank-30025>, Dezember 2019



- DB-Anwendungen: Alle Arten von Anwendungen (User Interface, Domänen-Anwendungen)
- Datenbankmanagementsystem (DBMS): Sammlung von ausführbaren Programmen, welche zur Umsetzung aller Zugriffe auf eine Datenbank eingesetzt werden
- Datenbank (DB): Sammlung strukturierter und dauerhaft gespeicherter Fakten für eine oder viele Domänenanwendungen
- Datenbanksystem (DS): Durch ein DBMS zugreifbare DB für eine oder viele Domänenanwendungen

Anforderungen an ein Database Management System (DBMS)

Funktion/Anforderung	Erklärung
Persistenz	Speicherung von Daten: elektronische Texte, Dokumente, ..., werden gespeichert, die durch Abfragen abgerufen werden können.
Verteilung	Verteilung der Daten auf verschiedene Speicherorte
Integrität	Regeln, die die logische Datenstruktur erhalten.
Recovery	Wiederherstellungsfunktion bei Datenverlust auf Anwenderseite
Gleichzeitigen Zugriff	Zugriffe durch viele Anwender gleichzeitig

Datenbankmodell

Ein Datenbankmodell illustriert die logische Struktur einer Datenbank

Ein Datenbankmodell umfasst nach Edgar F. Codd:

- Eine generische Datenstruktur, welche die Struktur einer Datenbank beschreibt (z.B. bei Relationalen Datenbanken: Relationen bzw. Tabellen)).
- Generische Operationen (z.B., Daten eintragen, ändern, abfragen)
- Integritätsbedingungen mit denen man die zulässigen Datenbankinhalte über die Grundstrukturen hinaus weiter einschränken kann (z.B. bei Relationalen Datenbanken: Typen, Schlüsselbeziehungen).

American National Standards Institute (ANSI)-3-Ebenenmodell



externe Ebene

Benutzeroberflächen, Datensichten,
API und Schnittstellen



konzeptionelle Ebene

Beziehungen, Daten



interne Ebene

Art und Form der Speicherung

Quelle:

<https://de.wikipedia.org/wiki/ANSI-SPARC-Architektur#/media/Datei:Drei-Ebenen-Schema-Architektur.svg>,

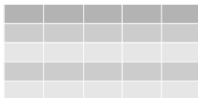
abgerufen am 12. April 2020.

Arten von Datenbankmodellen

Structured Query Language (SQL)/ Relational

Anfang 1980er
Sehr hohe
Verbreitung

relational



No not only Structured Query Language (No - SQL)

Mitte 1960er
Nahezu komplett
abgelöst

hierar-
chisch



Ende 1980er
Niedrige
Verbreitung

objekt-
orientiert



Anfang 2000er
mittlere
Verbreitung

document
store

```
data {  
  data1 {  
    "name": "data1",  
  },  
  data2 {  
    "name": "data2",  
  }  
}
```

Mitte 2000er
Relativ niedrige
Verbreitung

graph
store



Mitte 2000er
niedrige bis mittlere
Verbreitung

wide column
stores



Mitte 2010er
niedrige bis mittlere
Verbreitung

key-value
store



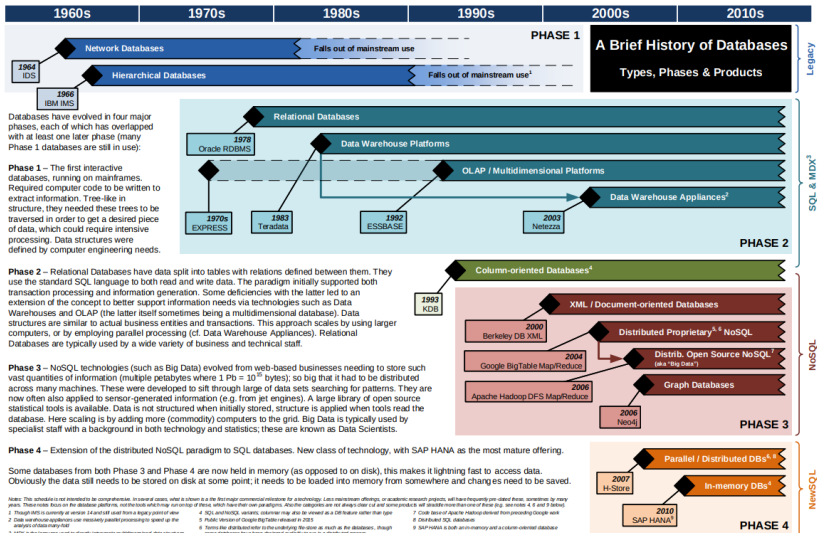
Vergleich Datenbankmodelle

Datenbankmodell	Entwicklung	Vorteile	Nachteile	Anwendungsgebiete	Bekannte Vertreter
Hierarchisch	1960er Jahre	Extrem schneller Lesezugriff, übersichtliche Struktur, technisch simpel	Starre Baumstruktur, die keine Verknüpfungen zwischen Bäumen zulässt	Banken, Versicherungen, Betriebssysteme	IMS/DB
Netzwerkartig	Anfang 1970er Jahre	Mehrere Suchwege zum Datensatz, keine strenge Hierarchie	Mangelhafte Übersicht bei größeren Datenbanken	Großrechner	UDS (Siemens), DMS (Sperry Univac)
Relational	1970	Einfache und flexible Erstellung und Bearbeitung, einfach erweiterbar, schnelle Inbetriebnahme, lebendige Wettbewerbssituation	Unhandlich bei großen Datenmengen, mangelhafte Segmentierung, künstliche Schlüsselattribute, externe Programmierschnittstelle, Objekteigenschaften und Objektverhalten schlecht abbildbar	Controlling, Rechnungswesen, Warenwirtschaftssysteme, Content-Management-Systeme, u. v. m.	MySQL, PostgreSQL, Oracle, SQLite, DB2, Ingres, MariaDB, Microsoft Access
Objektorientiert	Ende 1980er Jahre	Beste Unterstützung von objektorientierten Programmiersprachen, Speicherung multimedialer Inhalte	Zunehmend schlechtere Performance bei großen Datenmengen, wenige kompatible Schnittstellen	Inventar (Museen, Einzelhandel)	db4o
Dokumentenorientiert	1980er Jahre	Zentrale Speicherung von zusammengehörigen Daten in einzelnen Dokumenten, freie Struktur, multimediale Ausrichtung	Relativ hoher Organisationsaufwand, oft sind Programmierkenntnisse erforderlich	Webanwendungen, Internetsuchmaschinen, Textdatenbanken	Lotus Notes, Amazon SimpleDB, MongoDB, CouchDB, Riak, ThruDB, OrientDB

Ranking Datenbanksysteme

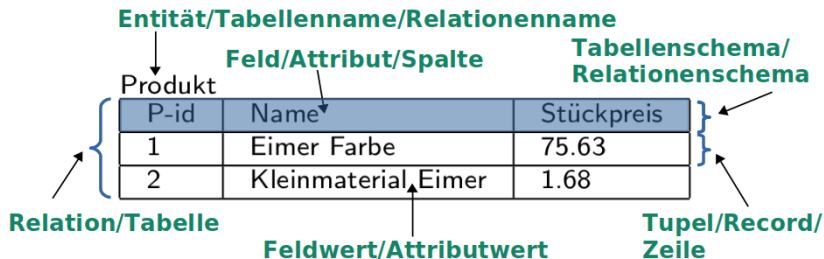
Rank Dec 2019	Rank			DBMS	Database Model	Score		
	Nov 2019	Dec 2018				Dec 2019	Nov 2019	Dec 2018
1.	1.	1.		Oracle	Relational, Multi-model	1346.39	+10.33	+63.17
2.	2.	2.		MySQL	Relational, Multi-model	1275.67	+9.38	+114.42
3.	3.	3.		Microsoft SQL Server	Relational, Multi-model	1096.20	+14.29	+55.86
4.	4.	4.		PostgreSQL	Relational, Multi-model	503.37	+12.30	+42.74
5.	5.	5.		MongoDB	Document, Multi-model	421.12	+7.94	+42.50
6.	6.	6.		IBM Db2	Relational, Multi-model	171.35	-1.25	-9.40
7.	7.	8.		Elasticsearch	Search engine, Multi-model	150.25	+1.85	+5.55
8.	8.	7.		Redis	Key-value, Multi-model	146.23	+1.00	-0.59
9.	9.	9.		Microsoft Access	Relational	129.47	-0.60	-10.04
10.	10.	11.		Cassandra	Wide column	120.71	-2.52	-1.10
11.	11.	10.		SQLite	Relational	120.36	-0.66	-2.65
12.	12.	12.		Splunk	Search engine	90.53	+1.46	+8.34
13.	13.	14.		MariaDB	Relational, Multi-model	86.79	+1.22	+9.53
14.	14.	15.		Hive	Relational	86.05	+1.83	+18.67
15.	15.	13.		Teradata	Relational, Multi-model	78.49	-1.86	-0.67
16.	16.	21.		Amazon DynamoDB	Multi-model	61.63	+0.26	+7.33
17.	17.	16.		Solr	Search engine	57.22	-0.56	-4.13
18.	19.	20.		SAP Adaptive Server	Relational	55.55	+0.25	-0.27
19.	18.	18.		FileMaker	Relational	55.14	-0.60	-1.51
20.	20.	19.		SAP HANA	Relational, Multi-model	54.17	-0.94	-2.14
21.	21.	17.		HBase	Wide column	53.46	-0.38	-6.55
22.	22.	22.		Neo4j	Graph	50.56	+0.03	+5.00
23.	23.	23.		Couchbase	Document, Multi-model	31.49	-0.49	-4.10
24.	24.	27.		Microsoft Azure Cosmos DB	Multi-model	31.43	-0.54	+8.06
25.	25.	25.		Microsoft Azure SQL Database	Relational, Multi-model	27.89	+0.27	+0.86
26.	26.	26.		Informix	Relational, Multi-model	25.51	-0.67	-1.42
27.	27.	33.		Google BigQuery	Relational	25.50	+0.06	+7.35
28.	28.	24.		Memcached	Key-value	24.46	-0.66	-5.15
29.	30.	29.		Firebird	Relational	22.44	+0.35	+2.39
30.	29.	28.		Vertica	Relational, Multi-model	22.17	-0.38	+0.79

Historie der Datenbanksysteme



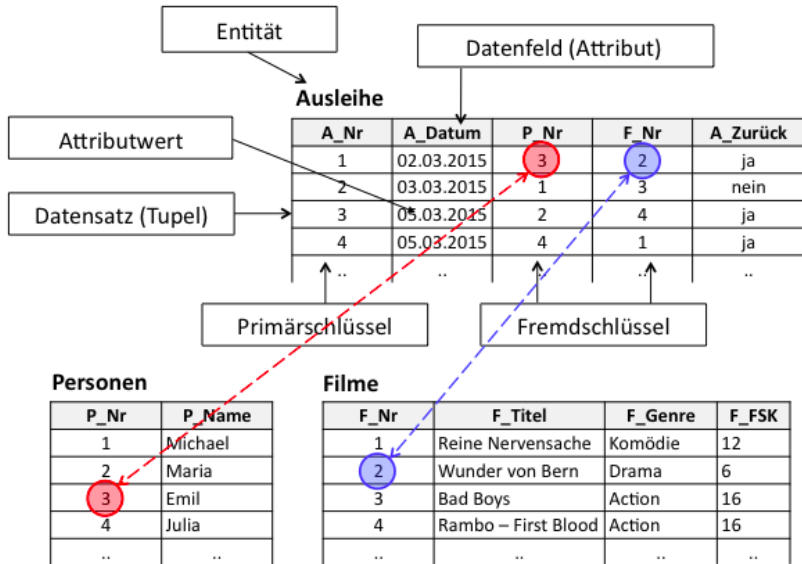
1. Überblick Datenbanken
2. Relationale Datenbanken
3. Dokumentenbasierte Datenbanken

Begriffe im Kontext einer Relation/Tabelle



- Relationenname: Name der Tabelle bzw. Relation oder Entität
- Attribute/Felder: Tabellenspaltenbezeichnung
- Tupel/Zeile/record: Tabellenzeile
- Attribut-/Felderwert: atomistisches Datenelement
- Relationenschema: Alle Attribute/Felder einer Tabelle und Ihre Typen/Relationendefinition

Relationales Datenmodell mit Primär- und Fremdschlüsseln



Modellierung innerhalb des Relationalen Datenbankmodells

Rechnung:

Abs: Mustermann GmbH • Hauptstrasse 123 • 13507 Berlin

Herr Dr. Hubert Brinkmann
Hauptstrasse 125/7/3
83395 Berlin
Deutschland

Rechnung

Mustermann Malerei GmbH.,
Hauptstrasse 123, 13507 Berlin
Tel/Fax +49 622 87654321
mustermann@mustermann.com
<http://www.mustermann.com>

Pos.	Bezeichnung	USt.	Menge	Preis	Gesamt
1	Farbe blau - Eimer 2,5 Liter	19%	10,00 Eimer	75,63 €	756,30 €
2	Kleinmaterial	19%		16,80 €	16,80 €
				Netto:	649,67 €
				19% USt:	123,43 €
				Brutto:	773,10 €

Relationales Datenmodell:

Produkt

P-id	Name	Stückpreis
1	Eimer Farbe	75.63
2	Kleinmaterial Eimer	1.68

Rechnung

R-id	P-id	K-id	F-id	Anzahl
1	1	1	1	10
2	1	1	1	10

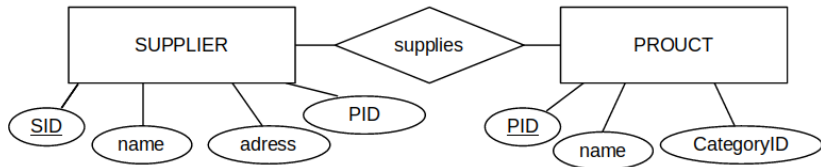
Kunde

K.-id	Name	Straße	Plz	Ort	Land
1	Dr. Hubert B.	Hauptstraße 125	83395	Berlin	DE

Firma

F-id	Name	Straße	Plz	Ort	Land
1	M. Malerei G.	Hauptstraße 123	13507	Berlin	DE

Quelle: <https://www.woax-it.com/Portals/0/Rechnungsvorlagen/Rechnungsvorlage.png>, Dezember 2019



- Informationssysteme
 - ▶ Operative Systeme
 - ▶ Managemententscheidungssysteme
 - ▶ Führungsunterstützungssysteme
 - ▶ Wissensdatenbank
 - ▶ Buchhaltungssysteme
 - ▶ Stammdatenpflege, ...

In den letzten Jahren haben sich für große Konzerne/große Datenmengen NO-SQL Datenbanken wie Wide Column Datenbanken und in-memory Datenbanken (im Hauptspeicher befindliche Datenbanken) neben RDBMS etabliert. Hauptverbreitung hat immer noch das RDBMS.

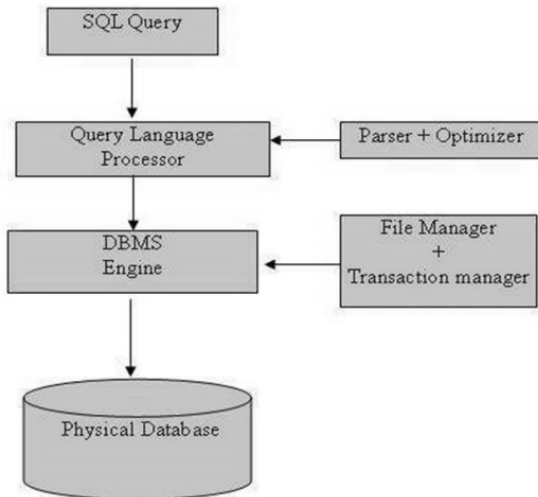
- Datawarehouse
- Web Applikationen (z.B. Stammdatenpflege)
- RDBMS-Hersteller: Oracle, Postgres, MySQL, SQLlight,...

→ Auch zu Zeiten von Big Data/ No-SQL hat das RDBMS seine klare Daseinsberechtigung und ist technologisch aktuell. Es ist gegenwärtig mit Abstand am meisten verbreitet. RDBMS kommt dort zum Einsatz, wo Datenintegrität besonders wichtig ist.

Structured Query Language (SQL)

- SQL ist eine Datenbanksprache, um Daten allumfassend verarbeiten zu können (Tabellen erzeugen, löschen, befüllen, auslesen, ...).
- SQL basiert auf einem mathematischen Konstrukt, der relationalen Algebra
- SQL ist synthaktisch (strukturell) und semantisch (von der Bedeutung her) an die englische Umgangssprache angelehnt.

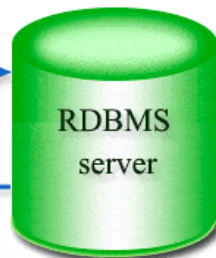
- `CREATE TABLE TABLE1 (ID INT, VALUE VARCHAR);`
- `DROP TABLE TABLE1;`
- `DELETE TABLE1;`
- `INSERT INTO TABLE1 VALUES (10, 'abc');`
- `UPDATE TABLE1 SET VALUE = 'cd' WHERE id = 10;`
- `SELECT VALUE FROM TABLE1 WHEREID > 20;`



Quelle: https://www.tutorialspoint.com/sql/sql_tutorial.pdf, Januar 2020

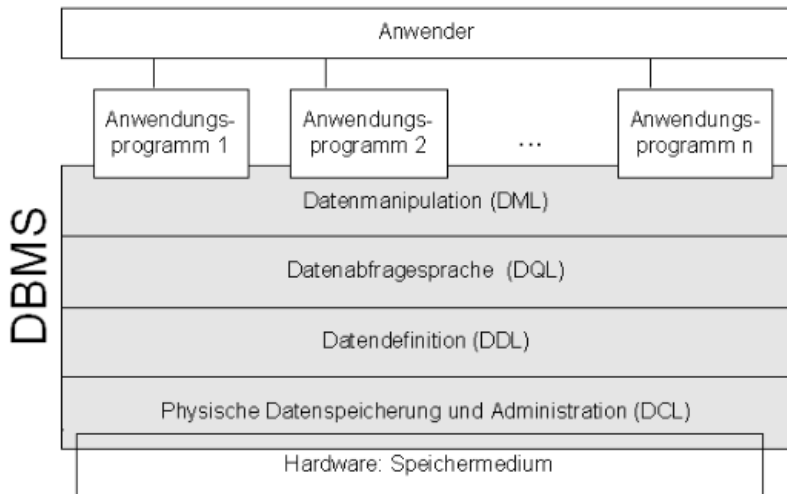
SQL statement

```
SELECT region_name  
FROM regions;
```

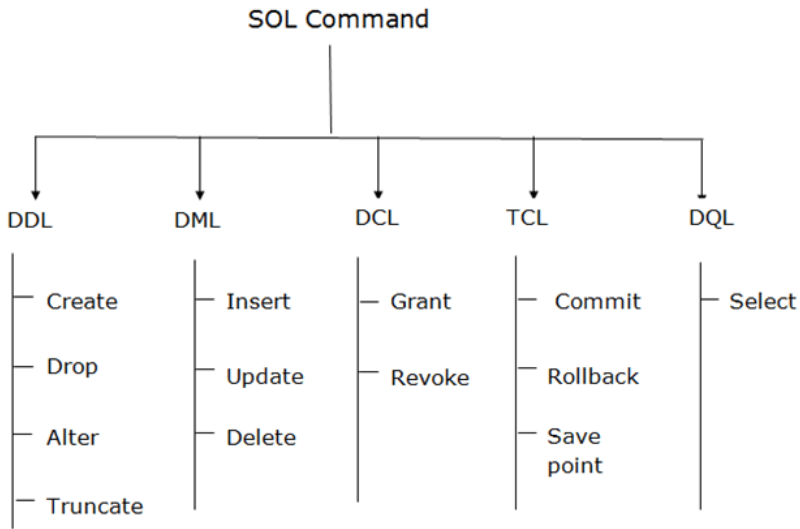
Statement sent to
RDBMS server

REGION_NAME
Europe
Americas
Asia
Middle East and Africa
row(s) 1 - 4 of 4

© w3resource.com



- DDL:** Data Definition Language: Definition/Erzeugung, Veränderung und Entfernung von Datenbankstrukturen/Datenbankschema
(CREATE, DROP, ALTER, TRUNCATE, RENAME, COMMENT)
- DCL:** Data Control Language: wird verwendet, um Berechtigungen zu vergeben und zu entziehen (GRANT, REVOKE)
- DQL:** Data Query Language: Daten aus der Datenbank lesen
(SELECT, JOIN)
- TCL:** Transaction Control Language: Arbeitet mit Transaktionen
(COMMIT, ROLLBACK, SAVE POINT, SET TRANSACTION)
- DML:** Data Manipulation Language:
(INSERT, UPDATE, DELETE, MERGE, CALL, EXPLAINPLAN, LOCKTABLE)



SQL-Befehl CREATE

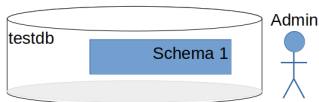
Der SQL-Befehl CREATE dient dazu, folgende Komponenten anzulegen:

- Datenbank
- Schema
- Datenbank-User
- Tabelle
- Index

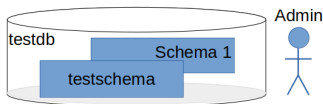
Ausdruck:	Bedeutung:
<code>CREATE DATABASE testdb;</code>	Erzeugt ein Datenbankschema
<code>CREATE SCHEMA testschema;</code>	Erzeugt ein Datenbankschema
<code>CREATE USER testuser;</code>	Erzeugt einen Datenbank User
<code>CREATE TABLE testtable (test_id INTEGER);</code>	Erzeugt eine Tabelle erzeugen
<code>CREATE INDEX idx_test ON testtable(name);</code>	Erzeugt Indizes auf einer Tabelle (für performante Suchabfragen)

Data Definition Language (DDL) - CREATE

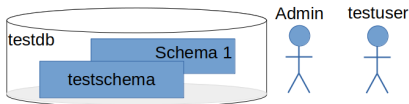
```
CREATE DATABASE testdb;
```



```
CREATE SCHEMA testschema;
```

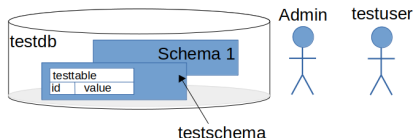


```
CREATE USER testuser;
```

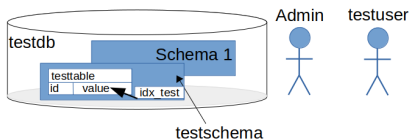


Data Definition Language (DDL) - CREATE

```
CREATE TABLE testtable ( test_id INTEGER);
```



```
CREATE INDEX idx_test ON testtable(name);
```



SQL-Befehl DROP

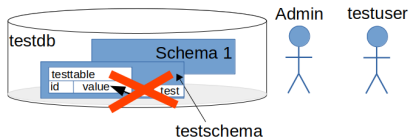
Der SQL-Befehl DROP dient dazu, folgende Komponenten aus dem DBMS zu entfernen:

- Datenbank
- Schema
- Datenbank-User
- Tabelle
- Index

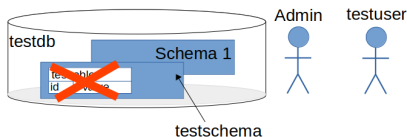
Ausdruck:	Bedeutung:
<code>DROP DATABASE testdb;</code>	Entfernt ein Datenbankschema
<code>DROP SCHEMA testschema;</code>	Entfernt ein Datenbankschema
<code>DROP USER testuser;</code>	Entfernt einen Datenbank User
<code>DROP TABLE testtable;</code>	Entfernt eine Tabelle erzeugen
<code>DROP INDEX idx_test;</code>	Entfernt Indizes auf einer Tabelle (für performante Suchabfragen)

Data Definition Language (DDL) - DROP

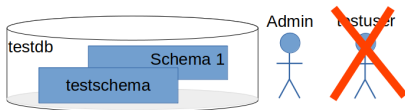
```
DROP INDEX idx_test;
```



```
DROP TABLE testtable;
```

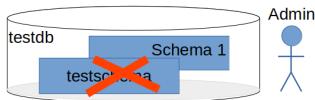


```
DROP SCHEMA testuser;
```

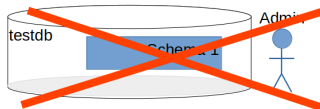


Data Definition Language (DDL) - DROP

```
DROP SCHEMA testschema;
```



```
DROP DATABASE testdb;
```



SQL-Befehl ALTER

Der Befehl ALTER ändert das Schema: Es können z.B. Spalten einer Tabelle hinzugefügt oder gelöscht werden (ALTER TABLE). Tabellen oder Spalten können umbenannt werden. Der Datentyp einer Spalte kann verändert werden (ALTER COLUMN).

Ausdruck:	Bedeutung:
ALTER TABLE testtable ADD COLUMN description VARCHAR;	Spalte hinzufügen
ALTER TABLE testtable DELETE COLUMN description;	Löscht Spalte description
ALTER TABLE testtable ALTER COLUMN description TYPE text;	Spaltentyp änderSpaltentyp ändern
ALTER TABLE testtable RENAME COLUMN description TO descr;	Benennt Spalte um
ALTER TABLE testtable RENAME TO test;	Benennt Tabelle um

Data Definition Language (DDL) - ALTER

- `ALTER TABLE testtable ADD COLUMN description VARCHAR;`
- `ALTER TABLE testtable DELETE COLUMN description;`

testtable		→	testtable		
id	value		id	value	description
INTEGER	INTEGER		INTEGER	INTEGER	VARCHAR

- `ALTER TABLE testtable ALTER COLUMN description TYPE text;`
- `ALTER TABLE testtable RENAME COLUMN description TO descr;`
- `ALTER TABLE testtable RENAME TO test;`

test		
id	value	descr
INTEGER	INTEGER	text

SQL-Befehl TRUNCATE

- TRUNCATE löscht den Inhalt einer Tabelle.
- TRUNCATE gehört zur DDL, da es die Tabelle zunächst löscht (DROP) und danach wieder mit allen Metadaten neu erzeugt.

Ausdruck:	Bedeutung:
TRUNCATE TABLE testtable;	Löscht den Inhalt der Tabelle

test		
id	value	descr
INT	INT	VARCHAR
1	2	"abc"
2	243	"def"
3	7	"ghi"
4	10	"ijk"
5	12	"lmn"



test		
id	value	descr
INT	INT	VARCHAR

SQL-Befehl DELETE

- DELETE löscht den Inhalt einer Tabelle.
- Anders als bei TRUNCATE, bleibt die Tabelle erhalten und wird Zeile für Zeile gelöscht. Deshalb gehört DELETE zur DML
- DELETE ist langsamer als TRUNCATE.

Ausdruck:	Bedeutung:
DELETE * FROM testtable (optional WHERE ...);	Löscht den Inhalt der Tabelle

test		
id	value	descr
INT	INT	VARCHAR
1	2	"abc"
2	243	"def"
3	7	"ghi"



test		
id	value	descr
INT	INT	VARCHAR

- nur für den Gebrauch innerhalb der Vorlesung -

SQL-Befehl INSERT

INSERT erzeugt eine oder mehrere Zeilen mit Inhalt in einer Tabelle.

Ausdruck:	Bedeutung:
<code>INSERT INTO testdb VALUES (1,2,'test');</code>	Importiert eine Zeile
<code>INSERT INTO testdb (description) VALUE ('abc')</code>	Importiert nur einen Feldinhalt
<code>INSERT INTO testdb VALUES (DEFAULT, 2, 'abc');</code>	Importiert eine Zeile, wobei das erste Feld vom Typ SERIAL ist (autoincrement)
<code>INSERT INTO testdb values (1,2,'abc'), (2,3,'def');</code>	Importiert mehrere Zeilen

Data Manipulation Language (DDL) - INSERT

- INSERT INTO testdb(description) VALUES (1,2,'test'), (2,4,'abc2');

test		
id	value	descr
INT	INT	VARCHAR

→

test		
id	value	descr
INT	INT	VARCHAR
1	2	test
2	4	abc2

SQL-Befehl UPDATE

UPDATE aktualisiert eins oder mehrere Felder einer oder mehrerer Spalten.

Ausdruck:	Bedeutung:
<code>UPDATE testtable set descr='123' WHERE descr='test';</code>	Updated Spalte descr mit dem Wert '123'
<code>UPDATE employees SET sales_count = sales_count + 1 WHERE id = (SELECT sales_person FROM accounts WHERE name = 'Acme Corporation');</code>	Erhöht sales_count in Tabelle employees um 1 für Verkäufer der Firma 'Acme Corporation' (von Tabelle accounts)

Data Manipulation Language (DDL) - UPDATE

- `UPDATE testtable set descr='123' WHERE descr='test';`

testtable		
id	value	descr
INT	INT	VARCHAR
1	2	test
2	4	abc2

→

testtable		
id	value	descr
INT	INT	VARCHAR
1	2	123
2	4	abc2

SQL-Befehle GRANT, REVOKE

GRANT erteilt Zugriffsberechtigungen für Rollen/User auf Datenbankobjekte wie

- Datenbanken
- Views
- Tabellen

Ausdruck:	Bedeutung:
<code>GRANT INSERT on testtable TO PUBLIC;</code>	Jeder User darf in testtable Daten einfügen
<code>GRANT ALL PRIVILEGES ON testdb TO user1;</code>	User user1 hat admin Rechte auf Datenbank testdb

Auswahl an möglichen Berechtigungen:

- SELECT: erlaubt das Auslesen aller oder spezieller Spalten einer Tabelle oder View.
- INSERT: erlaubt das Anlegen einer neuen Zeilen.
- UPDATE: erlaubt ein Update für eine speziellen Tabelle.
- DELETE: erlaubt das Löschen auf einer speziellen Tabelle.
- TRUNCATE: erlaubt eine spezielle Tabelle zu löschen.
- REFERENCES: erlaubt das Anlegen von Foreign Key Beziehungen.
- CREATE: erlaubt das Anlegen eines neuen Schemas für eine Datenbank.
- CONNECT: user darf sich mit Datenbank verbinden.
- TEMPORARY TABLE: erlaubt das Anlegen von Temporary Tables.
- ALL PRIVILEGES: alle Privilegien werden erteilt

SQL-Befehl REVOKE

REVOKE nimmt Zugangsgriffsberechtigungen für Rollen/User auf Datenbankobjekte wie

- Datenbanken
- Views
- Tabellen

zurück.

Ausdruck:	Bedeutung:
<code>REVOKE privileges ON object FROM user;</code>	privileges sind die GRANT Privileges, objects sind Datenbankobjekte
<code>REVOKE SELECT ON products FROM PUBLIC;</code>	Alle User dürfen kein SELECT mehr auf Tabelle products absetzen.

Transaction Controll Language (TCL) - COMMIT, ROLLBACK, SAVEPOINT

SQL-Befehle COMMIT, ROLLBACK, STOPPOINT

Transaktionen sind Folgen von SQL Anweisungen. Im Misserfolgsfall kann der Zustand der Datenbank auf einen definierten, stabilen Punkt zurückgesetzt werden. Transaktionen werden über COMMIT, ROLLBACK und SAVEPOINT definiert.

```
-- begin with transaction
BEGIN;
-- save point setzen
SAVEPOINT sp1;
INSERT INTO wines VALUES('Chateau Lafite 2003', '24');
ROLLBACK TO sp1;
UPDATE wines SET stock = stock + 24 WHERE winename = 'Chateau
Lafite 2003';
-- continue with other operations, and eventually
COMMIT;
```

Beispiel: Data Query Language (DQL) - SELECT

SQL-Befehl SELECT

SELECT dient dazu Tabelleninhalte zurückzugeben.

Ausdruck:	Bedeutung:
<code>SELECT * FROM testtable;</code>	Gibt alle Inhalte der Tabelle zurück
<code>SELECT * FROM testtable, testtable2;</code>	Gibt das Kreuzprodukt von testtable und testtable2 zurück
<code>SELECT description FROM testtable, testtable2;</code>	Gibt den Inhalt der Spalte description von testtable zurück
<code>SELECT id, description FROM testtable, testtable2;</code>	Gibt die Inhalte der Spalten id und description von testtable zurück.

Beispiel: Data Query Language (DQL) - SELECT

- SELECT * FROM KONTO, PRODUKT;

```
webshop=> select * from produkt;
```

1		2,12 €
6		2,30 €
7		3,40 €
8		1,40 €

```
webshop=> select * from konto;
```

1		0,00 €		DE2132141241
2		0,00 €		DE2132141241

```
webshop=> select * from konto,produkt  
;
```

1		0,00 €		DE2132141241		1		2,12 €
1		0,00 €		DE2132141241		6		2,30 €
1		0,00 €		DE2132141241		7		3,40 €
1		0,00 €		DE2132141241		8		1,40 €
2		0,00 €		DE2132141241		1		2,12 €
2		0,00 €		DE2132141241		6		2,30 €
2		0,00 €		DE2132141241		7		3,40 €
2		0,00 €		DE2132141241		8		1,40 €

Beispiel: Data Query Language (DQL) - SELECT

Ausdruck:	Bedeutung:
<code>SELECT * FROM testtable WHERE description='test';</code>	Die Zeilen von testtable werden zurückgegeben, bei denen Spalte description den Wert test annimmt.
<code>SELECT 3*7 AS RESULT;</code>	RESULT ist Alias und ergibt 21
<code>SELECT DISTINCT column1 FROM testtable;</code>	gibt den eindeutigen Inhalt (keine gleichen Feldinhalte) der Spalte column1 zurück
<code>SELECT column1,column2 FROM testtable ORDER BY column1 ASC;</code>	die Inhalte von column1 und column2 werden nach der Reihenfolge von column1 geordnet (ASC: aufsteigend, DSC: absteigend)
<code>SELECT column1,column2 FROM testtable GROUP BY column1;</code>	die Inhalte von column1 und column2 werden nach der nach Gleichheit von Spalte column2 gruppiert

Beispiel: Data Query Language (DQL) - SELECT

Aggregation mit select:

Ausdruck:	Bedeutung:
<code>SELECT MAX(umsatz) FROM verkauf;</code>	den Maximalwert des Umsatzes bestimmen
<code>SELECT AVG(alter) FROM kunde;</code>	das Arithmetische Mittel des Alters bestimmen
<code>SELECT SUM(transaktionen) FROM verkauf;</code>	die Summe der Transaktionen bestimmen

SQL-Befehl JOIN

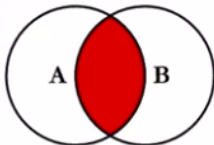
JOIN verbindet zwei Tabellen miteinander. Es gibt

- INNER JOIN: Schnittmenge
- LEFT JOIN: Linker Teil der zu verbindenen Mengen/Relationen
- RIGHT JOIN: Rechter Teil der zu verbindenen Mengen/Relationen
- FULL OUTER JOIN: Rechter und linker Teil der zu verbindenen Mengen/Relationen

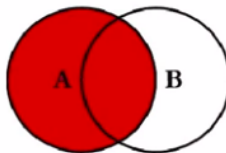
Ausdruck: z.B.

```
SELECT * FROM weather INNER JOIN  
          cities ON (weather.city= citites.name);
```

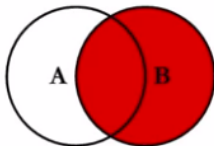
Data Query Language (DCL) - JOIN



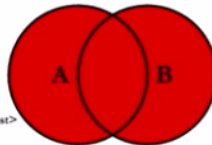
```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```

https://www.w3schools.com/sql/trysql.asp?filename=trysql_asc